



PDF Download
3772290.3772307.pdf
13 January 2026
Total Citations: 0
Total Downloads: 79

Latest updates: <https://dl.acm.org/doi/10.1145/3772290.3772307>

RESEARCH-ARTICLE

AASC: Activity-Adjusted Stake Consensus for Scalable IoT Blockchains

SUSMITA MONDAL, Indian Institute of Technology Jodhpur, Jodhpur, RJ, India

ADARSH RAJ SHRIVASTAVA, Indian Institute of Technology Jodhpur, Jodhpur, RJ, India

JATIN LOHAR, Indian Institute of Technology Jodhpur, Jodhpur, RJ, India

SUCHETANA CHAKRABORTY, Indian Institute of Technology Jodhpur, Jodhpur, RJ, India

Open Access Support provided by:

Indian Institute of Technology Jodhpur

Published: 06 January 2026

[Citation in BibTeX format](#)

ICDCN 2026: 27th International
Conference on Distributed Computing
and Networking
January 6 - 9, 2026
Nara, Japan

AASC: Activity-Adjusted Stake Consensus for Scalable IoT Blockchains

Susmita Mondal

Computer Science and Engineering, Ubisys Research
Group
INDIAN INSTITUTE OF TECHNOLOGY JODHPUR
Jodhpur, Rajasthan, India
p22cs203@iitj.ac.in

Jatin Lohar*

Computer Science and Engineering, Ubisys Research
Group
INDIAN INSTITUTE OF TECHNOLOGY JODHPUR
Jodhpur, Rajasthan, India
lohar.1@iitj.ac.in

Adarsh Raj Shrivastava*

Computer Science and Engineering, Ubisys Research
Group
INDIAN INSTITUTE OF TECHNOLOGY JODHPUR
Jodhpur, Rajasthan, India
shrivastava.10@iitj.ac.in

Suchetana Chakraborty

Computer Science and Engineering, Ubisys Research
Group
INDIAN INSTITUTE OF TECHNOLOGY JODHPUR
Jodhpur, Rajasthan, India
suchetana@iitj.ac.in

Abstract

Advances in decentralized decision-making have accelerated the adoption of the Internet of Things (IoT), with Blockchain emerging as a key distributed computing paradigm. It enables transparent multimodal data management without centralized infrastructure, ensuring traceability, trust, integrity, and authentication. Blockchain relies on consensus mechanisms validated by network participants, emphasizing the importance of fairness in these mechanisms, which is particularly critical for resource-constrained IoT systems with dynamically changing devices. To address this, we propose the *Activity-Adjusted Stake Consensus (AASC) mechanism*, a lightweight, iterative, and machine learning-augmented consensus protocol, specifically designed for Blockchain-enabled IoT. AASC continuously updates a reputation score (*Consensus Score*) for each IoT node, which reflects both historical and real-time node behaviors (*activity*), thereby enabling adaptive consensus even in a heterogeneous and ever-changing landscape. AASC is a novel approach that transcends conventional rule-based consensus, achieves remarkably low overhead with *linear $O(n)$ communication complexity* and *accessible $O(n \log n)$ computational complexity* per node, making it efficient for resource-constrained environments. Extensive experiments and comparative analysis demonstrate that AASC outperforms leading lightweight consensus protocols in terms of fairness, security, and scalability. The architecture effectively manages node churn and mitigates Sybil, collusion, and monopolization attacks by directly integrating randomized validator selection and swift penalization into the activity-based trust system. Thus, AASC substantially improves the reliability and practicality of Blockchain in dynamic IoT networks, paving the way for widespread adoption.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICDCN 2026, Nara, Japan
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1888-5/26/01
<https://doi.org/10.1145/3772290.3772307>

CCS Concepts

• **Computer systems organization** → **Sensor networks**; • **Security and privacy** → **Distributed systems security**; • **Computing methodologies** → **Machine learning**; • **Applied computing** → **Enterprise computing**.

Keywords

Blockchain, Consensus Protocol, IoT, Fairness, Reputation, Activity, Trust, Security

ACM Reference Format:

Susmita Mondal, Adarsh Raj Shrivastava, Jatin Lohar, and Suchetana Chakraborty. 2026. AASC: Activity-Adjusted Stake Consensus for Scalable IoT Blockchains. In *27th International Conference on Distributed Computing and Networking (ICDCN 2026)*, January 06–09, 2026, Nara, Japan. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3772290.3772307>

1 Introduction

The rapid proliferation of the Internet of Things (IoT) across domains such as transportation, manufacturing, and healthcare has created vast, heterogeneous networks that generate multimodal data at unprecedented scales [31]. Conventional cloud or edge-centric data management approaches face inherent limitations, including a single point of failure, privacy leakage, and weak trust guarantees [8]. Blockchain offers a decentralized alternative by providing tamper-evident storage, cryptographic security, and transparent governance for IoT data [31]. Through consensus mechanisms, Blockchain ensures network-wide agreement on transactions without centralized control, enabling secure, auditable, and fault-tolerant data exchange among heterogeneous IoT devices [8]. This synergy enhances interoperability, resilience, and stakeholder trust, establishing Blockchain as a promising foundation for secure and autonomous IoT ecosystems.

Despite these advantages, the implementation of Blockchain in IoT remains challenging due to the following factors:

- (1) **Resource Constraints:** Standard Blockchain protocols impose high computational and storage overhead, exceeding the capabilities of low-power IoT devices such as wearables and embedded sensors [7].

- (2) *Consensus Inefficiency*: Classical consensus mechanisms (e.g., Proof of Work [22], Proof of Stake [16]) exhibit high latency, high energy consumption, and limited scalability, making them unsuitable for dynamic IoT environments [7].
- (3) *Network Dynamics and Scalability*: IoT networks experience frequent node churn, and heterogeneous device capabilities degrade the stability and security of existing Blockchain platforms [8].
- (4) *Predictability of Block Producers*: Rule-based consensus protocols often expose deterministic block producer selection, increasing the risk of targeted attacks and reducing fairness [32].
- (5) *Interoperability Limitations*: Current Blockchain-as-a-Service (BaaS) solutions (e.g., Helium [22], IOTA [7], IoTeX [22]) provide specialized functionality but fail to universally address heterogeneous IoT requirements, leading to fragmented and non-optimal deployments [32].

To address these critical limitations, we propose a novel consensus framework, *Activity-Adjusted Stake Consensus* (AASC), specifically designed for the dynamic and resource-constrained nature of IoT environments. Unlike traditional approaches that rely on static stake values or slowly evolving reputation scores, AASC introduces a real-time, activity-weighted trust mechanism integrated with probabilistic validator selection. This coupling enables the system to dynamically prioritize trustworthy nodes based on recent behavior, while preserving decentralization and minimizing computational overhead.

At its core, AASC leverages machine learning (ML)-driven behavior analytics to model trust as a constrained ranking problem, enabling lightweight yet adaptive validator selection without sacrificing fairness, system integrity, or scalability. The protocol is optimized to operate efficiently under node churn, heterogeneous device capabilities, and adversarial conditions, offering a scalable blueprint for secure Blockchain consensus in large-scale IoT deployments.

The key contributions of this work are as follows:

- We reformulate consensus as a rank-based learning problem and implement a lightweight supervised ML model to compute activity-adjusted trust in real time.
- We design a consensus protocol tailored for volatile IoT environments, with native support for frequent device joining, leaving, and adaptive network reconfiguration.
- Through extensive experimentation, we demonstrate that AASC outperforms existing IoT-focused BaaS consensus mechanisms across multiple metrics, including throughput, latency, CPU, and memory usage, offering superior resistance to adversarial manipulation.

The remainder of the paper is structured as follows. Section 2 reviews related work on Blockchain-based IoT consensus. Section 3 defines the system model and design assumptions. Section 4 introduces the AASC protocol, along with its mathematical foundations. Section 5 analyzes the correctness, efficiency, and security properties of AASC. Section 6 presents experimental results and comparative evaluations. Finally, Section 7 concludes the paper.

2 Related Work

Blockchain-as-a-Service (BaaS) has emerged as a promising solution for addressing key IoT challenges such as privacy, trust, fault tolerance, and data management [29]. Despite its advantages, Blockchain

integration in IoT faces significant hurdles, including scalability limitations, energy inefficiency, and latency overheads [9]. The existing BaaS platforms operated by IBM [12], AWS [12], and Microsoft Azure [12], as well as customized networks such as Algorand [3] and Tendermint [2], offer varying levels of support for IoT applications by enabling flexible deployment and integration of business logic. Recent research efforts [14, 15] focus on customizing these platforms for heterogeneous IoT environments by enhancing interoperability, securing communication, protecting identities, and fostering trust through reputation systems. In general, advances in BaaS-based IoT can be categorized into various directions, including the following:

2.1 Network Topology and Scalability:

Dorri et al. [5] introduced an overlay Blockchain-IoT network that enables the removal of aged transactions and summary-based storage, thus reducing memory usage while maintaining conventional consensus, which has limited behavioral adaptivity. Yang et al. [29] proposed a deep reinforcement learning-based sharding framework for the Industrial IoT (IIoT) Blockchain, dynamically clustering nodes to increase throughput and synchronization. However, their primary focus is on sharding efficiency; node trust, fairness, or dynamic validator selection remain unaddressed. Elsaadany et al. [9] presented multi-tier self-scaling clusters for transaction management, reducing block validation latency through layered verification. While facilitating network decentralization and auto-scaling, this work employs rule-based node allocation, offering limited adaptivity to rapidly changing trust and adversarial conditions. Dorri et al. [6] proposed a lightweight scalable Blockchain (LSB) that uses time-based randomized block managers to enhance throughput, reduce off-chain storage, and improve anonymity. However, the validator election leans on static randomness.

2.2 Block Structure and Security Enhancements:

Dorri et al. [4] proposed the Tree-chain, a tree-structured ledger that assigns hash-based validation ranges to improve transaction partitioning and throughput. However, their fundamental consensus is not adaptive to dynamic node behavior in IoT scenarios. Ramanathan et al. [21] described Proof-of-Real-time Transfer (PoRT), a protocol that utilizes selective validators and transaction randomness, along with integrity-testing messages, to reduce computational load and improve the honesty of validators. Their security analysis did not reveal the challenges of scalability in node churn scenarios. Fan et al. [10] presented Digital Twin as a virtual IoT proxy, implementing access control and trust institutions to reinforce privacy, but they refrained from exploring real-time consensus or validator dynamics.

2.3 Reputation-Driven and Adaptive Consensus:

Zhang et al. [30] designed a fair proof-of-reputation protocol, selecting consensus groups based on reputation and executing Practical Byzantine Fault Tolerance (PBFT)-style decisions. Its reputation updating is epoch-based and is not tied to real-time node behaviors. Kleinrock et al. [13] introduced Proof-of-Reputation (PoR) with a Nakamoto fallback block proposal, where the power of the proposal is tied to statically earned reputation, revealing lags in adversarial detection. Kara et al. [11] proposed Proof-of-Chance (PoCh), a randomized chance-based validator selection intended for dynamic topologies. This work overlooks node activity or behavior, which may lead to

fairness issues. Zhao et al. [32] described the Proof-of-Evolutionary Model (PoEM), an ML-augmented protocol that offers low latency, which inspired our proposed model. However, Blockchain updates are still tied to periodic adjustments in this work, rather than continuous, node-level behavioral analytics. Putrat et al. [18] and Vyzovitis et al. [27] explore validator ranking and PubSub-based dissemination, with scoring models and probabilistic trust, but with limited real-time adaptability. Michalopoulos et al. [15] integrated a verifiable random function and a reputation voting protocol into Hyperledger Fabric, targeting permissioned environments. However, the system assumes a static externally provisioned reputation. Other protocols [20, 26] are also explored, focusing on reputation-weighted and fairness-based approaches, but with mostly theoretical security and limited scalability, paying little attention to node churn and real-time adaptation.

Despite recent advances, existing BaaS-based IoT consensus protocols exhibit critical limitations: (i) most rely on static or epochal trust models [13, 30], hindering rapid responsiveness to node churn or adversarial scenarios; (ii) they insufficiently adapt to real-time node activity, often ignoring behavioral analytics in favor of randomness [6, 11]; (iii) few leverage ML for live trust scoring [18, 32], limiting their resilience to manipulation and heterogeneity; (iv) scalability and efficiency are challenged by PBFT-like frameworks [30], which become inoperable as network size grows; and (v) empirical evidence is typically constrained to small-scale or static simulations, with limited validation under dynamic, real-world IoT conditions [21]. To address these gaps, we introduce AASC, a consensus protocol that continuously integrates historical and real-time node behaviors into a trust score, allowing instant, granular penalization, and resilience against Sybil, collusion, and manipulation attacks. AASC achieves accessible computational ($O(n \log n)$) and linear communication ($O(n)$) complexity through lightweight, pre-trained ML integration, and uniquely supports message duplication management and explicit flow regulation. Extensive simulations confirm superior throughput, latency, resource efficiency, and security, establishing a new benchmark for decentralized, fair, and scalable consensus in dynamic IoT networks.

3 System Design of AASC

AASC is a hybrid consensus mechanism that proceeds in two different phases. It incorporates a simplified Proof-of-Stake (PoS)-based bootstrap phase during initialization, which establishes a lightweight and stable trust baseline over the first six blocks. This transition ensures that early consensus is computationally efficient while enabling long-term fairness, particularly in the presence of frequent node churn. Then, there is a fully activity-adjusted phase that governs subsequent block production. This design ensures stability in achieving Blockchain agreement by incorporating validator behavior, quantified in terms of activity. AASC adaptively modulates incentives based on historical participation, promoting trustworthiness, and discouraging manipulation. The key terminology and feature selection rationale used in AASC are summarized in Table 1. Overall, AASC mitigates block producer predictability and enhances chain stability, even in volatile IoT environments.

3.1 Problem Formulation:

To formally define the AASC protocol, we model the consensus process as a supervised learning problem within a dynamic Blockchain

environment. The goal is to predict validator reliability based on their activity history and system attributes, thereby enabling efficient, fair, and secure validator selection in resource-constrained IoT networks.

We begin by considering the Blockchain system as a sequence of state transitions, where the current system state is based on validated transactions. In this setting, each IoT node operates under volatility, frequently joining or leaving the network, and exhibits a set of static (e.g., hardware-related) and dynamic (e.g., behavioral) features. These features are used to assess validator quality over time. Let the dataset for training $\mathcal{DT} = \{data_1, data_2, \dots, data_v\}$ represent a vectorized collection of node features. Each record includes:

- Transactional features (e.g., size, type, and frequency of transactions),
- Behavioral features (e.g., number of blocks produced, penalty history), and
- A binary label indicating whether the node acted as a block producer in the observed epoch (label = 1) or not (label = 0).

Using this labeled dataset, AASC trains a lightweight supervised ML model to (i) learn feature weights for computing Consensus Score (CS), (ii) rank validators based on probabilistic block-producing potential, and (iii) penalize validators with poor performance or failure to participate over multiple epochs. This predictive formulation enables AASC to adaptively select validators that are not only trustworthy but also responsive to system dynamics, thus balancing efficiency, fairness, and fault tolerance.

3.2 Overview of the System:

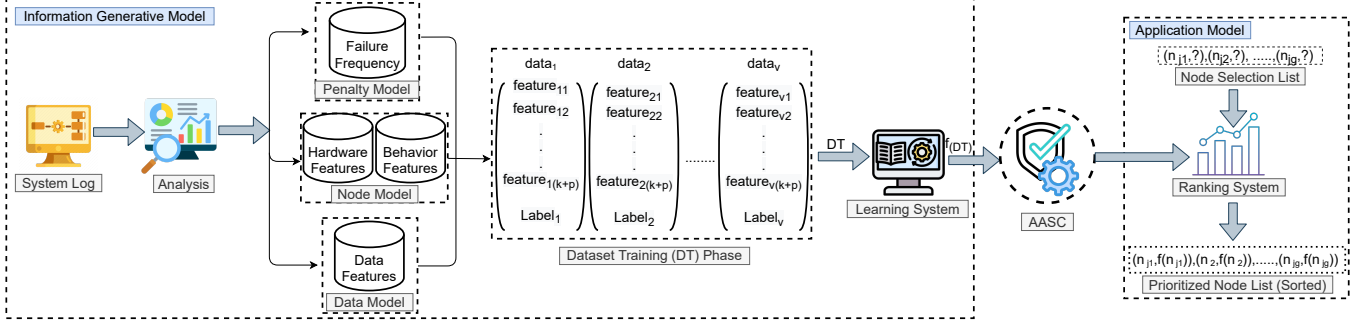
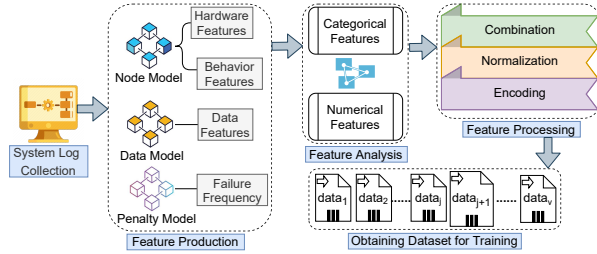
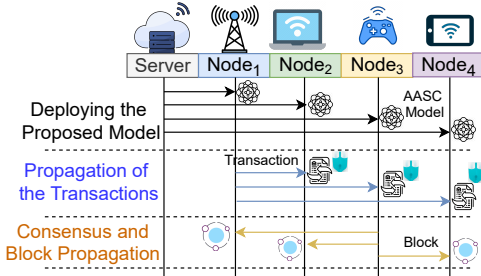
The AASC system operates in two stages: model generation and model application, as illustrated in Figure 1. Given the limited computational resources of the IoT nodes, model training and updates are typically transferred [25] to a higher-capacity entity, such as an edge server or a federated learning setup [17]. At the same time, inference and consensus decisions are performed locally on the Blockchain network. The system collects and preprocesses raw node features from operational logs (Figure 2), which are categorized into:

- A data model, containing unprocessed content hashes, hardware metrics (e.g., transaction latency), and behavioral metrics (e.g., number of blocks successfully mined),
- A penalty model, tracking failure frequency (e.g., missed block production opportunities).

These features form the basis of the training dataset ($\mathcal{DT}$), which is used in the supervised learning model that predicts CS for each node. Notably, this training process is customizable and dataset-agnostic, supporting a broad range of feature definitions. Once trained, the model ranks nodes in a prioritized validator list based on their historical reliability and probabilistic performance. Prioritized validators are then randomly selected for block production in each round. Upon successful block mining and broadcast, the network updates its global state, recalculates CS values, and begins a new epoch with an updated priority list, thereby ensuring a continuous and adaptive consensus process. This activity-aware, learning-driven validator selection mechanism distinguishes AASC from conventional approaches by enabling lightweight, real-time adaptability to node dynamics without compromising system integrity.

Table 1: Terminologies and Feature Selection Rationale in AASC Algorithm

Abbreviation	Expanded Expression	Feature Selection Rationale
LIT (lit)	Last Inter-staking Time (time since the last block was mined)	Measures recent activity and current network engagement
AIST (aist)	Average Inter-staking Time (average time between blocks mined)	Low AIST reflects steady behavior, serving as long-term commitment
CST (cst)	Cumulative Stake Time (total time a node had its stake active)	Quantifies reliability, higher CST values indicate honest participation
CBM (cbm)	Consecutive Blocks Mined (number of consecutive blocks mined by the node with binary representation)	Prevents validator monopolization and promotes fairness
TST (tst)	Total Stake Time across all validators (cumulative time the nodes have had their stakes active)	Provides network-wide normalization for fair consensus score computation and comparison

**Figure 1: The Conceptual Diagram of the Activity-Adjusted Stake Consensus****Figure 2: Process of Data Collection****Figure 3: Application Process of Consensus Mechanism**

4 Activity-Adjusted Stake Consensus (AASC)

The AASC protocol comprises three core components: (i) Broadcasting and transaction dissemination (Algorithm 1), (ii) Consensus Score (CS) calculation and validator ranking (Algorithm 2), and (iii) Block mining and finalization (Algorithm 3). The initialization phase of the AASC (PoS-based bootstrapping) helps collect the necessary system logs for validator activity metrics (Table 1), enabling consensus training. Labeled data with reliable nodes (characterized by high uptime, low error rates, and consistent behavior) are prioritized, and features are selected based on activity correctness, ensuring that frequency alone does not contribute to the CS learning process. This introduces randomization to diverse representative data. Security is maintained by verifying sample data through digital signatures and authenticated

validator ports. Only validated datasets are used for training, and the final model is deployed across all nodes.

When a new node joins the network, it is granted fair participation by being evaluated for inclusion in the validator list based on its activity and system logs. Although some rounds of incremental training may be required before a node becomes a block producer, this ensures robustness against cold-start bias and supports node churn. Figure 2 outlines the data preparation pipeline, which involves: (i) System log collection, (ii) Feature extraction (e.g., hardware, behavioral, data features), (iii) categorization, (iv) Feature normalization and encoding, and (v) Dataset preparation for model training. The matrices used for this adjustment summarizing activity patterns, failure rates, and historical performance are detailed in Table 1 and Table 2, which guide the reward and penalty system in the validator selection process.

4.1 AASC Execution Workflow:

The AASC execution workflow is illustrated in Figure 3, encompassing the entire consensus round in a BaaS-based IoT setting. The system operates as follows:

- (1) A central server or gateway locally trains the AASC model using system logs and then disseminates the trained model to all participating IoT nodes off-chain.
- (2) Each node locally verifies the model (a one-time process) and integrates it into the consensus engine.
- (3) When a transaction is initiated (e.g., by *Node1*), it is broadcast to peer nodes (*Node2*, *Node3*, *Node4*, etc.).
- (4) Upon receiving the transaction, all nodes execute the AASC model to evaluate the CS scores and rank the validators.
- (5) A validator is randomly selected from the top-ranked nodes based on CS probabilities, and it proceeds to mine the block and broadcast the result to the network.
- (6) The system updates the global state and recalculates the CS values for the next epoch.

Algorithm 1 Broadcasting (For Connecting Block Producer)

Require: Message with parameters: type, message_id, data

```

1: if message[ 'message_id' ] is already received then
2:   return // Stops if the message was already processed
3: else
4:   Mark message[ 'message_id' ] as received
5: end if
6: Initialize connection with peers
7: Send message to peers
8: if message[ 'type' ] is 'store validators' then
9:   Wait to receive data from peers
10:  Decode the received data and parse as response
11:  Update list of validators by adding validators from
    response[ 'validators' ]
12: else if message[ 'type' ] is 'new transaction' then
13:   Wait to receive acknowledgements from the peers
14: end if
15: Close the socket connection

```

AASC supports variable transaction volumes and hardware heterogeneity, enabling context-sensitive selection using time-windowed transaction data. In scenarios where a node experiences temporary disconnection or failure, its CS is maintained by neighboring peers and resynchronized upon rejoining the network, addressing node churn. To handle memory constraints in IoT devices, older ledger data is summarized using cryptographic hashes, with full records offloaded to a secure storage and retrievable when required. Although the current model training is centralized for efficiency, consensus execution remains fully decentralized, where each node independently runs the inference locally using the shared model. This modular and adaptive design ensures that AASC remains lightweight, resilient to node churn, and equitable throughout the set of validators, making it highly suitable for dynamic and decentralized IoT environments.

4.2 Broadcasting and Transaction Dissemination:

The Algorithm 1 describes the initiation of consensus through the transmission of messages used by a block producer (validator) in a BaaS-based IoT environment. The algorithm connects several block producers and ensures that critical messages, such as validation updates and transaction notifications, are disseminated efficiently without redundancy. As a transaction (message) is received, the algorithm verifies if it has already been processed with the help of the *message_id* identifier. If processed previously, the node stops processing to handle duplication and ensure idempotency (lines 1-2). If not processed previously, the nodes mark the transaction as received and initialize a socket connection with the neighboring (peer) nodes (lines 3-7). Depending on the message type, the broadcasting characteristics change. If the message is *store validators*, the node expects a response from the peers that contains the current validator identities (lines 8-11). Conversely, if the type is *new transaction*, the node awaits acknowledgement to confirm the successful delivery of the message (lines 12-13). Upon completing the necessary exchanges based on the message type, the connection with the neighboring nodes is closed (lines 14-15). The algorithm enhances message handling extensibility in terms of its types and contributes to the robustness of inter-node communication in the BaaS-based IoT environment powered by AASC.

Algorithm 2 Consensus Score Calculation

Require: Validator parameters params containing *aist*, *cst*, *tst*, *lit*, *cbm*
Ensure: Consensus Score (CS)

```

1: Retrieve aist, cst, tst, lit, cbm from params (as in Table 1)
2: if cst = 0 then
3:   Set aist  $\leftarrow 2 \times$  number of validators in Blockchain
4: else if cst = 1 then
5:   Set aist  $\leftarrow$  number of validators in Blockchain
6: end if
7: Compute CS (as in Table 2)
8: return CS

```

4.3 Consensus Score (CS) Calculation:

The algorithm 2 formalizes the calculation of the consensus score (CS), which utilizes a quantitative metric to assess the activity of the validators and their effectiveness in decentralizing the AASC mechanism. The algorithm takes the parameter set *params* as input (line 1), with the components *aist*, *cst*, *tst*, *lit*, and *cbm* described in Table 1. The score is formalized in two stages, where *aist* is adjusted based on trust and the state of *cst*. If the node is untrusted (*cst* = 0), *aist* is doubled relative to the number of validators in the system that initiate the penalty (lines 2-3), and if trusted (*cst* = 1), it is set to the validator count baseline (lines 4-5). The final score is calculated and stored in CS; the mathematical expressions are explained in Table 2. The CS calculation locally incurs a constant $O(1)$ computation cost, whereas the CS calculation for all validators has a cost of $O(n)$. The algorithm balances various factors: prioritizing the validators with low latency and high trust states, and quadratically suppresses the CS to penalize nodes with frequent behavior misalignment and inconsistency. The logarithmic term ensures a positive avalanche effect in the CS score, accompanied by a small improvement in latency, enhancing resiliency.

4.4 Block Mining and Validator Ranking:

The algorithm 3 presents the block mining approach, which helps dynamically and fairly determine the next block producer in a BaaS-based IoT environment, typically suitable for mining in a permissioned setup. The unified selection algorithm mitigates monopolization and integrates validator behavioral history with randomness and activity levels, thereby promoting fairness. Each validator executes the mining process independently for every block interval (every round or every epoch), following certain processes.

4.4.1 Initialization: The process requires parameters (described in Table 2 and Algorithm 3) and PoS fallback for the early stages (lines 4-7) of chain formation (six blocks), creating sufficient historical data for AASC-based activity score (CS) calculation. If the parameters of a validator are set to *None* (uninitialized), they are assigned the default values, which differ from those of the current validator to distinguish them (lines 1-3).

4.4.2 Activity: After the availability of sufficient blocks, the algorithm 3 identifies who mined the block in position $block_pos = length(Blockchain) - 6$ as a validator (lines 8-9). It guarantees a dynamic, yet temporally bounded, scoring window. During this phase, the pointer moves to a position denoted by $n - 6$ (where n is the current length of the Blockchain), providing stability by supplying sufficient time for the block to mature [24]. The chain pointer focuses on minimizing risks related to chain forking and

Algorithm 3 AASC Mining (occurs simultaneously at each validator node)**Require:** Optional message**Ensure:** A new block mined (if eligible) and broadcast it

```

1: if parameters is None then
2:   Initialize parameters with default values
3: end if
4: if Length of Blockchain  $\leq 6$  then
5:   Call POS(message) // Proof-Of-Stake fallback phase
6:   return
7: end if
8: Set block_pos  $\leftarrow$  length of Blockchain - 6
9: Set mined_by  $\leftarrow$  validator that mined block at block_pos
10: Update parameters for mined by:
    Update AIST as the average inter-staking time:
      Calculate aist (as described in Table 2)
    Increment CST:   cst  $\leftarrow$  cst + 1
    Update TST as total stake time for the validator:
      tst  $\leftarrow$  block_pos + 1 - self.active
    Set LIT to the current block position:
      lit  $\leftarrow$  block_pos + 1
    Set CBM to 1, indicating a newly mined block
11: for each validator port in network do
12:   if port  $\neq$  mined_by then
13:     Set CBM  $\leftarrow$  0 for port, indicating no consecutive block was mined
14:   end if
15: end for
16: Calculate Consensus Scores (CS) for each validator using self.score()
17: Generate a random number in range  $[1, 2^{64}]$  and broadcast it to other validators.
18: Receive random numbers from other validators.
19: Perform bitwise XOR of all random numbers to generate global_random
20: Divide global_random by  $2^{64}$ .
21: Sort validators by CS and calculate cumulative sum array CS_cumul
22: Normalize CS_cumul to range  $[0, 1]$  by dividing each value by the total sum of CS
23: Assign each validator to bins based on CS_cumul:
    • Validator  $v_1$  is assigned to range  $(0, CS\_cumul_1]$ 
    • Validator  $v_i$  is assigned to range  $(CS\_cumul_{i-1}, CS\_cumul_i]$ .
24: Identify bin containing global_random
25: Set block_producer  $\leftarrow$  validator corresponding to the identified bin
26: if node_port = block_producer then
27:   Generate new block and reset pending transactions
28:   Broadcast new block
29: end if

```

supports reorganization through its design choice. AIST reflects the average waiting time (block distance) between successive blocks mined, where CST is incremented to denote the active stake of a particular node (lines 10). TST is computed as the total time the nodes had their stakes active during the validation period (lines 10). LIT updates the time for the last mined block (lines 10), and CBM indicates whether a validator has currently mined a block (lines 11-15).

4.4.3 Election: After validating states of all validators, the algorithm 3 computes CS for every validator, which prioritizes high-scoring validators and penalizes validators with short staking histories, high latency, or misbehavior (line 16). Then, to improve security, AASC incorporates randomness in validator selection, masking their identity to prevent predictability. Each validator generates a random

number within the range of $[1, 2^{64}]$, and the global random seed combines all the random numbers of all validators using the bitwise XOR. The resulting value is normalized to the range $[0, 1]$ divided by 2^{64} (lines 17-20). The normalized random number is used to modify the CS. It is sorted according to the node address for deterministic ordering (line 21), with a computational complexity of $O(n \log n)$. The scores are then transformed into a cumulative sum array (CS_{cumul}), scaling it to the range $[0, 1]$ by dividing each value by the total sum of the CS values (line 22). Through the distribution function, sorted CS creates disjoint probability bins, which help determine the next block producer. We divide the range $[0, 1]$ of (CS_{cumul}) into N bins, where N is the number of validators. Suppose that the validator v_1 is assigned to the range $[0, CS_{cumul,1}]$, v_2 to $[CS_{cumul,1}, CS_{cumul,2}]$, etc. for N validators. Next, we use the normalized random number from the XOR operation to identify the corresponding bin. The validator associated with the selected bin is chosen as the new block producer (lines 23). The probabilistic approach of selecting high-scoring validators as block producers might be predictable. However, the AASC approach ensures the choices, but adds unpredictability, masking the choices of validators and promoting fairness (lines 24-25).

4.4.4 Broadcasting: If the current validator is selected as the block producer, it generates a new block by collecting transactions on a time window basis, eliminating duplicated *message_id*. Then, the block producer appends it to their local chain, clears the transaction pool to prevent redundancy, and broadcasts the new block to their neighboring peers (lines 26-29).

5 Discussion and Analysis

Communication is a critical aspect of AASC, which operates on IoT devices with limited computational and network resources. We intend to build a lightweight, scalable protocol for low latency, even if it requires continuous broadcasting. Duplicate messages are filtered out by message identifiers, minimizing redundant network usage and reducing communication overhead, making it suitable for BaaS-based IoT devices. The design of the AASC mechanism offers several benefits, including activity metrics (Table 1), which facilitate equitable participation by all validators and promote fairness. The reward and penalty system prevents monopolization and validates block producers with continuous activity monitoring, encouraging honesty (Table 2). The dynamic joining and leaving of nodes is handled discreetly, with the maintenance of their continuous CS based on previous historical behaviors (activity).

5.1 Correctness Analysis:

The correctness of the proposed AASC protocol demonstrates its ability to function as a secure and efficient consensus mechanism suitable for dynamic Blockchain-based IoT ecosystems. It is understood that formal properties, which guarantee reliability and trustworthiness among decentralized nodes, are widely known as agreement, validity, termination, safety, and liveness [28]. The agreement, validity, and termination are more focused on operational and observable commitments in classical distributed computing. In contrast, safety and liveness provide high-level theoretical guarantees on system behavior in all possible circumstances, including faults, adversarial actions, or network partitions [23].

Table 2: Algorithm Formulation for AASC Mechanism

Terminologies	Description	Purpose	Mechanism	Impact	Remarks
AIST (aist)	The average waiting time between block production.	Rewards validators who mine less frequently but participate consistently.	$aist \leftarrow \frac{aist \times cst + (block_pos + 1 - lit)}{cst + 1}$	- Prioritizes nodes with longer waiting periods between block productions. - Promotes fairness by giving higher scores to less frequent but steady participants.	Validators' historical activity metrics are updated after every successful block production.
CS (Consensus Score)	The consensus score calculation for each validator.	Evaluating the reliability and effectiveness of validators.	$CS = \frac{aist \cdot (1 - \frac{cst}{tst}) \cdot \log\left(1 + \frac{1}{lit + 10^{-6}}\right)}{(cbm + 1)^2}$	- Promotes validators with low latency and high trust sates. - Penalize frequent behavioral misalignment and inconsistencies.	Logarithmic term ensures score suppression for misbehaving nodes quadratically.
Matrix Updating Formulas Seriatim					
Abbreviation	Description	Purpose	Mechanism	Impact	Remarks
$1 - \frac{cst}{tst}$	Balances participation between older and newer nodes.	Penalizes excessive staking dominance to ensure fairness among validators.	Node Participation: • New nodes: $\frac{cst}{tst} \approx 0 \rightarrow (1 - \frac{cst}{tst}) \approx 1$ • Old nodes: $\frac{cst}{tst} \approx 1 \rightarrow (1 - \frac{cst}{tst}) \approx 0$	Newer nodes score higher, encouraging broader network participation.	The trust ratio penalizes instability by reflecting fluctuation over time.
$\log\left(1 + \frac{1}{lit + 10^{-6}}\right)$	Calculates recent network activity.	Encourages faster participation after inactivity, with diminishing returns.	Activity Evaluation: • Active nodes: LIT small $\rightarrow \log\left(1 + \frac{1}{lit}\right)$ larger $\rightarrow$ higher score • Inactive nodes: LIT large $\rightarrow \log\left(1 + \frac{1}{lit}\right)$ smaller $\rightarrow$ lower score	Rewards active nodes while maintaining fairness in the score.	In the equation 10^{-6} is added to prevent division by zero.
$(cbm + 1)^2$	Penalizes consecutive block mining.	Reduces the score for validators producing multiple consecutive blocks.	- Set to 1 indicating the validator just mined a block. - All other validators' CBM values are reset to 0	Prevents monopolization by a single validator.	Indicates double-signing or protocol violation.

5.1.1 Agreement: AASC employs the broadcasting mechanism (Algorithm 1), which effectively suppresses duplicate messages among the validator set of size n . A global random number is generated via the XOR aggregation of $O(n)$ uniformly distributed 64-bit random values (Algorithm 3, lines 17-20). This yields a combined randomness space of size 2^{64} , ensuring an almost negligible probability of collision (less than 10^{-18} per consensus round as $\frac{1}{2^{64}} \approx 5.4 \times 10^{-20}$) [19]. Validators then compute their CS in $O(n)$ time complexity and deterministically perform cumulative binning to select the leader (Algorithm 3, lines 23). Because all honest nodes follow the identical deterministic procedure, the probability that honest validators disagree over the block producer is negligible, thus robustly enforcing agreement.

5.1.2 Validity: Block producers must be legitimate validators with positive consensus scores for validity. The score formula (for calculating CS) integrates dynamic real-time metrics that reflect the stake, activity, and recent behavior of the validator (Algorithm 2). Default values scale linearly with validators n (for example, $aist = 2n$ when the stake is zero), while the quadratic penalty in the denominator significantly decreases the score for consecutive block mining attempts. It ensures that malicious, inactive, or newly created validators, such as Sybils [1], have scores tending to zero, effectively excluding them from selection (Table 2). Hence, only valid blocks from trustworthy validators are generated, satisfying the validity property.

5.1.3 Termination: Each consensus round involves the exchange of 64-bit random numbers among all n validators in $O(n)$ messages and the transmission of blocks with acknowledgements (Algorithm 1). CS calculation also is $O(n)$ per validator (Algorithm 2). The protocol design includes explicit broadcast timeouts and retry limits, which prevent indefinite waiting or deadlock conditions. As a result, the protocol guarantees the termination of consensus decisions in a finite time proportional to $O(n)$.

5.1.4 Safety: The safety property is upheld by enforcing the selection of a single leader per round through the XOR randomness and weighted binning scheme (Algorithm 3), eliminating conflicting block proposals. Moreover, the quadratic penalty applied to the extracted metric of consecutive blocks, (cbm) , drastically reduces the score of a validator if it tries to produce multiple sequential blocks, with the score dropping by at least a factor of $\frac{1}{(cbm+1)^2}$. For example, mining only two consecutive blocks reduces the score by a factor of four, thus disincentivizing monopolistic behavior. Coupled with duplicate message filtering and broadcast acknowledgement workflows, this prevents forks and double-signing, maintaining block lengths bounded by six (stable) blocks in the longest chain rule [24].

5.1.5 Liveness: The liveness of the AASC protocol is ensured by the dynamic nature of the CS and the leader election process. Validators with positive CS, calculated based on real-time activity and stake

balance terms, maintain a non-zero probability of selection at each round, ensuring steady block production opportunities. XOR-based joint randomness is inherently unpredictable and resistant to manipulation by adversaries, preventing scenarios where node churn or network asynchrony could stall progress. Consequently, the protocol accommodates nodes joining or leaving the network without blocking forward progress, guaranteeing continuous Blockchain growth.

5.2 Security Analysis:

The security of AASC arises from the novel combination of random leader selection, activity-based trust metrics, and punitive mechanisms for suspected misbehavior, as formally described in Algorithms 1–3. Here, we detail these assurances using established correctness and adversarial models, carefully referencing how each property is achieved within our consensus architecture.

5.2.1 Unpredictable Leader Selection: As proved in Algorithm 3, the random leader election process is accomplished by each node generating an independent random value, which bitwise XOR collaboratively aggregates among validators. The selected block producer is then uniquely determined by mapping the resulting joint randomness to the cumulative CS bins. As shown in lines 16–23 of Algorithm 3, this process is:

- **Unbiasable:** No single validator can control jointly generated randomness without compromising the majority of nodes.
- **Unpredictable:** The probabilistic mapping ensures future leaders cannot be feasibly inferred in advance (Algorithm 3).

5.2.2 Stake Centralization (51%) Attack Mitigation: The algorithm 2 incorporates the term cbm quadratically reducing the CS of validators that attempt to mine consecutive blocks. Furthermore, any node that mines consecutive blocks is penalized (penalty factor $\geq \frac{1}{4}$) and deprioritized for future selection. If extended control is detected, the node's CS is suppressed for a defined period, in line with the defence frameworks outlined for stake-based systems (modeled within our unique CS metric and bins).

5.2.3 Anti-Sybil and Collusion Resistance: Our design (Table 2) takes advantage of the dimensions of cst and the historical activity ($aist$). As only nodes with established positive activity can accrue non-negligible CS, newly introduced Sybil identities cannot rapidly gain block production rights, as outlined in the node management mechanics (Algorithm 2, lines 2–8; Algorithm 3, lines 8–10). Colluding nodes cannot easily participate in the leader selection process due to the randomization and score regularization steps.

5.2.4 Replay, Double-Signing, and Fork Resistance: The algorithm 1 strictly tracks unique *message_ids* for all consensus and block events, stifling attempts at replays, stale injections, or duplicative block claims. Furthermore, by organizing consensus around a single block producer per round and encoding finality as a function of stable blocks ($n-6$), AASC substantially limits fork risk and double-spending windows, paralleling security treatments like modern BFT and PoS chains.

5.2.5 Partition Tolerance and Cluster Robustness: If the network partitions, the protocol gracefully forms independent clusters, each with its intra-cluster consensus, but with a clear process for reconciliation and block stability upon rejoining, guaranteeing that no partition

Table 3: State-of-the-Art Consensus Comparison

Consensus	Mechanism	Comp Cost	Comm Cost
PoS	Stake-based	$O(b)$	$O(n)$
PoEM	Supervised ML Model-based	$O(f \cdot d)$	$O(n)$
PoCh	Hash-based	$O(n \log n)$	$O(n)$
AASC	Model and Hash-based	$O(n \log n)$	$O(n)$

a) Comp $\rightarrow$ Computation, b) Comm $\rightarrow$ Communication

can finalize conflicting transactions unless a supermajority collusion occurs (which is penalized via instant CS adjustment).

AASC has unique deterministic mapping from CS and joint randomness to leader identity. The strict message processing in Algorithm 1 prevents forks and double signing. The penalty mechanism in Algorithm 2 ensures that staking centralization cannot persist, aligning with correctness criteria for modern PoS or Delegated PoS (DPoS) algorithms. Even if in a set of n validators running Algorithms 1–3 has no more than $\lfloor n/2 \rfloor$ validators actively adversarial in a round, the algorithm still functions correctly. Notably, leader precomputation, stake centralization, Sybil, replay, and fork attacks are formally enforced by the mechanisms defined in AASC algorithms.

5.3 Time Complexity Analysis:

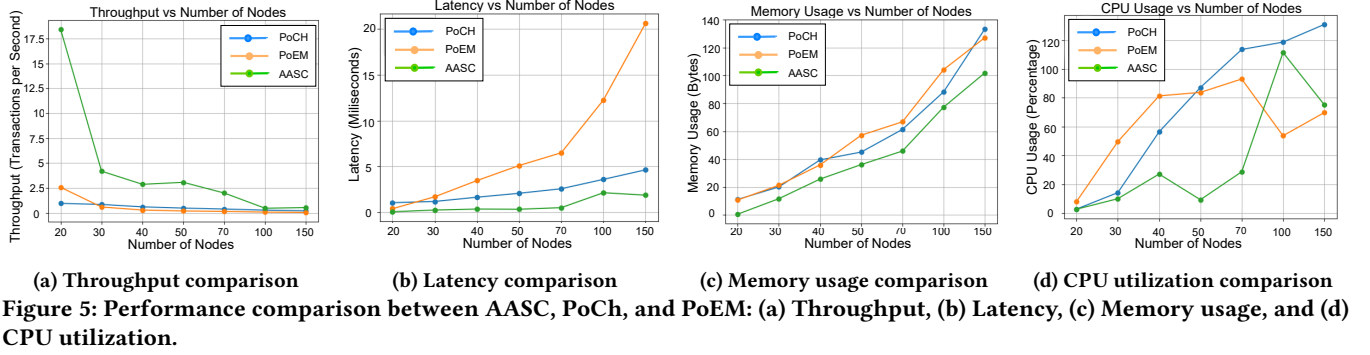
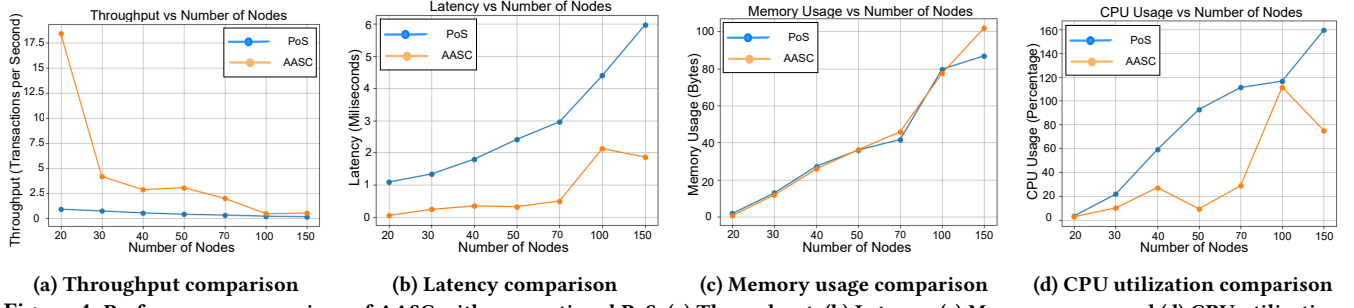
BaaS-based IoT consensus protocols have various factors that contribute to their computational and communication complexity. We theoretically compare AASC with the three most prominent state-of-the-art consensus mechanisms in Table 3. The computation cost refers to the resource consumption of each node when processing the protocol to reach an agreement that is supported by the entire system. The communication cost aims to demonstrate the convergence time required for network connectivity and message broadcasting to achieve consensus. Here, n represents the total number of active nodes in the system, and b represents the stake, which helps to accelerate the consensus process in PoS. $O(f \cdot d)$ in PoEM consensus refers to the number of features (f) and model complexity (d) for the number of layers hidden in model inference. The PoCh involves sorting and computing hashes to a small candidate list in logarithmic scale ($O(n \log n)$) and broadcasting it with ($O(n)$) cost, which is a requirement for convergence. AASC has the same communication costs as all other consensus protocols (i.e., $O(n)$) in Table 3. However, the computation cost is relatively low ($O(n \log n)$), including sorting, CS validation, and the bin selection process. The computational cost can be scaled to $O(n)$ by balancing probabilistic selection, which will be addressed in the future. AASC is suitable for BaaS-based IoT ecosystems because the computation and communication costs are quite affordable even when n is larger.

6 Performance Evaluation

We experimentally evaluated AASC from a BaaS-based IoT environment and accumulated the performance results. Applying the activity matrices discussed in Tables 1 and 2, we estimate the efficiency of our proposed methodology and demonstrate the applicability of comparing AASC to state-of-the-art consensus mechanisms, such as PoS, PoEM, and PoCh.

6.1 Experimental Setup:

We experimented with a well-defined and controlled ecosystem to obtain reproducible and impartial results. We also provide all the necessary hardware specifications to simulate and test AASC for



evaluation purposes. The experiments are conducted on a cluster of machines (virtually mimicking Blockchain nodes) with similar hardware capabilities to simulate a distributed IoT environment and gather performance results. The hardware specifications are configured with an AMD Ryzen 5 3500U with a Radeon Vega Mobile Gfx 2.10 GHz processor, ~18 GB of usable memory, 118 GB SSD storage, and a 64-bit Microsoft Windows operating system. The different approaches are implemented using the Python IDE, and the dataset ($\mathcal{DT}$) is trained randomly using a normal distribution. We comprehensively compare various parameters of the AASC with those of the PoS, PoEM, and PoCh consensus, including throughput, latency, memory, and CPU utilization. We assume a basic construction of the consensus algorithms with a difficulty level comparable to AASC and adjust the parameters accordingly, repeating the experiments with 150 nodes. Moreover, by averaging out the results, we ensure scalability, where the number of nodes can be increased.

6.2 Performance Metrics:

The following metrics are considered to evaluate the performance of AASC and compare it with the POS, PoEM, and PoCh: (i) *Throughput* (the number of transactions processed per second), (ii) *Latency* (the average time taken to achieve consensus for a transaction), (iii) *Memory usage* (the memory consumption observed from the validator nodes under the consensus workload, and (iv) *CPU utilization* (the percentage of CPU resource utilization by the nodes during the consensus mechanism).

We compare these four baseline approaches with three state-of-the-art consensus mechanisms as follows:

- *PoS*: Proof-of-Stake, where a node that owns more "stake" preferably gets the appropriate hash value set as the target [16].

- *PoEM*: Proof of Evolutionary Model utilizes supervised ML to efficiently determine block producers [32].
- *PoCh*: Proof of Chance creates random hash-based opportunities to determine the selection and candidacy of the miner [11].

The comparison is differentiated into two echelons, where the PoS vs. AASC is the first direction; as PoS is used to initiate the AASC model for primary rounds, it is necessary to compare the performance of these two mechanisms for subsequent rounds. The second direction is to compare AASC with PoEM and PoCh, two closely related consensus mechanisms that must be differentiated from our methodology. The finding highlights that AASC balances fairness and resource efficiency, providing better scalability, as demonstrated in Figures 4 and 5.

6.3 Result Analysis:

We analyze the results based on two criteria: we evaluate the efficiency of any consensus algorithm in terms of throughput and latency, and then assess the applicability of the consensus mechanisms in terms of memory and CPU utilization. The confidence interval for each point represents the average of 10 runs, where the error bar shows the standard deviation rate ± 1 . In Figure 4, the blue line represents PoS, the orange line represents AASC, while in Figure 5, the blue line represents PoCh, the orange line represents PoEM, and the green line represents AASC.

6.3.1 AASC vs PoS. AASC exhibits higher throughput as the number of nodes increases, whereas PoS throughput shows a linear relationship. Figure 4a shows that AASC can handle a significant amount of tps (transactions per second) under the stress of scalability. Similarly, AASC exhibits low and stable latency, reaching consensus faster (Figure 4b), demonstrating its suitability for BaaS-based real-time IoT.

AASC is more memory-efficient (as shown in Figure 4c) and exhibits a performance trend similar to PoS. AASC maintains relatively low and predictable CPU usage, making it a lightweight and scalable solution, as shown in Figure 4d. The drop in AASC throughput is due to the increased XOR aggregation latency as more nodes broadcast random numbers simultaneously, which we will optimize in the future.

6.3.2 AASC vs PoCh and PoEM. Figure 5a shows a higher throughput of AASC than PoEM and PoCh (which remain lower and relatively similar). AASC maintains better performance as the number of nodes grows, even if throughput decreases, and remains superior to PoEM and PoCh. AASC has the lowest latency, showing impressive responsiveness in Figure 5b. Evident in Figure 5c, AASC balances memory usage and increases performance, making it scalable. For CPU utilization in Figure 5d, comparatively, AASC is more stable, moderate, and consistent, but PoEM performs better beyond 85 nodes. The decrease in CPU utilization is attributed to the better amortization of ML inference costs across longer block times as the network expands, which reduces the spikes in per-node processing. This will be addressed in the future.

The results demonstrate the adaptability of AASC in both smaller and larger-scale, resource-constrained, and dynamic BaaS-based IoT systems, offering substantial scalability. AASC improves efficiency and applicability compared to the baseline consensus mechanism, as it consistently performs for more than a hundred nodes. The framework is stable and offers higher throughput and lower latency, making it ideal for real-time applications such as sensor-based systems, autonomous vehicles, and smart grids.

6.4 Open Challenges and Future Directions:

Although AASC demonstrates strong simulated performance, several directions require further investigation. Rigorous analysis of BFT bounds, timeout parameterization, and failure recovery under adversarial conditions remains open. A comprehensive empirical evaluation, particularly in real-world IoT deployments with high node churn rates, is crucial to validate scalability. The centralized off-chain training server, although efficient, represents a potential bottleneck; future work will explore federated learning and decentralized model aggregation. Protocol specifications for model synchronization and versioning conflict resolution require formalization. Finally, a holistic analysis of amortized overhead from auxiliary training-server interactions (such as asynchronous log uploads and model downloads) and optimal retraining schedules would provide deeper end-to-end performance insights.

7 Conclusion

In this paper, we propose the Activity-Adjusted Stake Consensus (AASC) protocol. This lightweight and scalable consensus mechanism utilizes hash-based bins and a trained model initialization to ensure validator fairness in BaaS-based IoT ecosystems. We formulated an activity-based incentive and penalty mechanism to optimize the protocol, which depends on the node's past and present behavior, accurately handling the dynamism of IoT nodes. In addition, we compare AASC with state-of-the-art consensus mechanisms and achieve captivating results by averaging hundreds of simulations. We plan to address the minor limitations of our proposed consensus mechanism in order to enhance efficiency and simplify the initialization stage. We

also aim to incorporate edge devices to assess network performance in a distributed environment, thereby broadening and improving the applicability of our solution. The framework effectively manages resources, striking a balance between speed and security, thereby mitigating the competing demands of a modern and sustainable BaaS-based IoT ecosystem.

Acknowledgments

This work is partially supported by MeitY, Govt. of India, through the project S/Meity/SKS/20210139 at IIT Jodhpur.

References

- [1] Alex Biryukov and Daniel Feher. 2020. ReCon: Sybil-resistant consensus from reputation. *Pervasive and Mobile Computing* 61 (2020), 101109.
- [2] Ethan Buchman. 2016. *Tendermint: Byzantine fault tolerance in the age of blockchains*. Ph. D. Dissertation. University of Guelph.
- [3] Jing Chen and Silvio Micali. 2016. Algorand. *arXiv preprint arXiv:1607.01341* (2016).
- [4] Ali Dorri and Raja Jurdak. 2021. Tree-chain: A lightweight consensus algorithm for iot-based blockchains. In *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–9.
- [5] Ali Dorri, Salil S Kanhere, and Raja Jurdak. 2019. MOF-BC: A memory optimized and flexible blockchain for large scale networks. *Future Generation Computer Systems* 92 (2019), 357–373.
- [6] Ali Dorri, Salil S Kanhere, Raja Jurdak, and Praveen Gauravaram. 2019. LSB: A Lightweight Scalable Blockchain for IoT security and anonymity. *J. Parallel and Distrib. Comput.* 134 (2019), 180–197.
- [7] Evaldas Drasutis. 2022. IOTA smart contracts.
- [8] Romaric Duvignau, Michel Raynal, and Elad Michael Schiller. 2024. Self-stabilizing byzantine multivalued consensus. In *Proceedings of the 25th International Conference on Distributed Computing and Networking*. 12–21.
- [9] Riham Elsaadany and Guy Bégin. 2023. A blockchain node organizing and auto-scaling architecture for the Internet of Things. In *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 36–43.
- [10] Xinxin Fan, Qi Chai, Lei Xu, and Dong Guo. 2020. Diam-iot: A decentralized identity and access management framework for internet of things. In *Proceedings of the 2nd ACM international symposium on blockchain and secure critical infrastructure*. 186–191.
- [11] Mostefa Kara, Abdelkader Laouid, Mohammad Hammoudeh, Muath AlShaikh, and Ahcène Bounceur. 2022. Proof of Chance: A Lightweight Consensus Algorithm for the Internet of Things. *IEEE Transactions on Industrial Informatics* 18, 11 (2022), 8336–8345. doi:10.1109/TII.2022.3168747
- [12] Abhirup Khanna, Anushree Sah, Vadim Bolshev, Alessandro Burgio, Vladimir Panchenko, and Marek Jasiński. 2022. Blockchain–cloud integration: A survey. *Sensors* 22, 14 (2022), 5238.
- [13] Leonard Kleinkrock, Rafail Ostrovsky, and Vassilis Zikas. 2020. Proof-of-reputation blockchain with nakamoto fallback. In *International Conference on Cryptology in India*. Springer, 16–38.
- [14] Xiulong Liu, Zhiyuan Zheng, Hao Xu, Zhelin Liang, Gaowei Shi, Chenyu Zhang, and Keqiu Li. 2025. Enabling Consistent Sensing Data Sharing Among IoT Edge Servers via Lightweight Consensus. *IEEE Trans. Comput.* (2025).
- [15] Fotis Michalopoulos, Sokratis Vavilis, Harris Nivavis, and Konstantinos Loupos. 2024. Integrating a Hybrid Lightweight Consensus Algorithm in HyperLedger Fabric. In *International Summit on the Global Internet of Things and Edge Computing*. Springer, 188–203.
- [16] Cong T Nguyen, Dinh Thai Hoang, Diep N Nguyen, Dusit Niyato, Huynh Tuong Nguyen, and Eryk Dutkiewicz. 2019. Proof-of-stake consensus mechanisms for future blockchain networks: fundamentals, applications and opportunities. *IEEE access* 7 (2019), 85727–85745.
- [17] Dinh C Nguyen, Ming Ding, Quoc-Viet Pham, Pubudu N Pathirana, Long Bao Le, Aruna Seneviratne, Jun Li, Dusit Niyato, and H Vincent Poor. 2021. Federated learning meets blockchain in edge computing: Opportunities and challenges. *IEEE Internet of Things Journal* 8, 16 (2021), 12806–12825.
- [18] Guntur Dharma Putrat, Sidra Malik, Volkan Dedeoglu, Salil S Kanhere, and Raja Jurdak. 2023. Trust and reputation management for blockchain-enabled iot. In *2023 15th International Conference on COMMunication Systems & NETWORKS (COMSNETS)*. IEEE, 529–536.
- [19] Zhan Qin, Yin Yang, Ting Yu, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2016. Heavy hitter estimation over set-valued data with local differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 192–203.
- [20] Mohammad Saidur Rahman, Ibrahim Khalil, Mohammed Atiquzzaman, and Abdelaziz Bouras. 2025. A lightweight practical consensus mechanism for supply chain blockchain. *High-Confidence Computing* 5, 1 (2025), 100253.

- [21] Narayanan Ramanathan, Ragul Kumar, Siddharth Banerjee, and Sriram Padmanabhan. 2023. Proof of Real-Time Transfer: A Consensus Protocol for Decentralized Data Exchange. In *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–9.
- [22] Abhimanyu Rawat, Vanesa Daza, and Matteo Signorini. 2023. Not Enough IoT After All: Visiting Transactional Characteristics of IoT Blockchains.. In *DLT*.
- [23] Syamsul Rizal and Dong-Seong Kim. 2025. Enhancing Blockchain Consensus Mechanisms: A Comprehensive Survey on Machine Learning Applications and Optimizations. *Blockchain: Research and Applications* (2025), 100302.
- [24] Roberto Saltini. 2022. BigFoot: A robust optimal-latency BFT blockchain consensus protocol with dynamic validator membership. *Computer Networks* 204 (2022), 108632.
- [25] Youliang Tian, Ta Li, Jinbo Xiong, Md Zakirul Alam Bhuiyan, Jianfeng Ma, and Changgen Peng. 2021. A blockchain-based machine learning framework for edge services in IIoT. *IEEE Transactions on Industrial Informatics* 18, 3 (2021), 1918–1929.
- [26] Sokratis Vavilis, Harris Niavis, and Konstantinos Loupos. 2025. A Fair and Lightweight Consensus Algorithm for IoT. *arXiv preprint arXiv:2503.08607* (2025).
- [27] Dimitris Vyzovitis, Yusef Napora, Dirk McCormick, David Dias, and Yiannis Psaras. 2020. Gossipsub: Attack-resilient message propagation in the filecoin and eth2.0 networks. *arXiv preprint arXiv:2007.02754* (2020).
- [28] Jie Xu, Cong Wang, and Xiaohua Jia. 2023. A survey of blockchain consensus protocols. *Comput. Surveys* 55, 13s (2023), 1–35.
- [29] Xinjie Yang, Tianyi Xu, Fengbiao Zan, Tao Ye, Zhaofang Mao, and Tie Qiu. 2023. An overlapping self-organizing sharding scheme based on DRL for large-scale IIoT blockchain. *IEEE Internet of Things Journal* 11, 4 (2023), 5681–5695.
- [30] Tao Zhang and Zhigang Huang. 2023. FPoR: Fair proof-of-reputation consensus for blockchain. *ICT Express* 9, 1 (2023), 45–50.
- [31] Yao Zhao, Youyang Qu, Yong Xiang, Yushu Zhang, and Longxiang Gao. 2023. A lightweight model-based evolutionary consensus protocol in blockchain as a service for IoT. *IEEE Transactions on Services Computing* 16, 4 (2023), 2343–2358.
- [32] Yao Zhao, Youyang Qu, Yong Xiang, Yushu Zhang, and Longxiang Gao. 2023. A Lightweight Model-Based Evolutionary Consensus Protocol in Blockchain as a Service for IoT. *IEEE Transactions on Services Computing* 16, 4 (2023), 2343–2358. doi:10.1109/TSC.2023.3238690